# IB113 Úvod do programování a algoritmizace

Cvičení 5

Jaromír Plhák






# Řetězec (string)

- Sekvence znaků (čísla, písmena, symboly) uzavřených:
  - a) V apostrofech: my_string = 'Brian'
  - b) V úvozovkách: my_string = "Brian"
- Pokud má řetězec obsahovat apostrof?

```
>>> print('I'm Brian !')
SyntaxError : invalid syntax
>>> print('I\'m Brian !') # Escaping
I'm Brian !
>>> print("I'm Brian !")
I'm Brian !
```

# Délka řetězce

- Funkce len():

```
>>> example = "Python"
>>> len(example)
6
```

- Prázdný řetězec = prázdné apostrofy / uvozovky:

```
>>> empty = ""
>>> type(empty)
<class 'str '> # It really is a string
>>> len(empty)
0
```

# Jednotlivé znaky řetězce

- Jednotlivé znaky řetězce jsou indexované celými čísli
  - Indexuje se od 0 po len(string) - 1!

**>>> example = "Python"**

| **Znak** | P | y | t | h | o | n |
|---|---|---|---|---|---|---|
| **Index** | 0 | 1 | 2 | 3 | 4 | 5 |

- Syntaxe zpřístupnění znaků: string[index]

**>>> character = example[1]**

**>>> character**

**'y'**

- Řetězce jsou neměnitelné (*immutable*): nefunguje např. example[1] = 'i'!

# Rozsah indexů (řezy)

| Znak | P | y | t | h | o | n |
|---|---|---|---|---|---|---|
| Index | 0 | 1 | 2 | 3 | 4 | 5 |

```
>>> example [1:4] # starting + ending index
'yth'
>>> example [1:] # starting only , default ending index is the length of string
'ython'
>>> example [:4] # ending only , default starting index is 0
'Pyth'
```

# Rozsah indexů (řezy)

| Znak | P | y | t | h | o | n |
|---|---|---|---|---|---|---|
| Index | 0 | 1 | 2 | 3 | 4 | 5 |

```
>>> example [ -4: -1] # starting and ending
'tho'
>>> example [ -4:] # starting only
'thon'
>>> example [: -1] # ending only
'Pytho‘
>>> example [::2] # string[start:end:step]
'Pto'
```

# Iterace nad řetězcem

- Cyklus for

```
string = "spam!"
for character in string:
    print (character)
```

- Proměnná character postupně nabývá hodnoty všech znaků řetězce
- Druhá možnost:

```
string = "spam!"
for i in range(len(string)):
    print(string[i])
```

# Aritmetické operátory nad řetězcem

- +

```
>>> "King " + "Arthur " + str (1) + "st"
'King Arthur 1st'
```

- *

```
>>> "Five" * 3
'FiveFiveFive'
```

# Operátory in, not in

- Test na výskyt znaku / podřetězce v retězci

```
>>> 'p' in "Python"
False
>>> 'r' not in "Python"
True
>>> 'tho' in "Python"
True
>>> 'toh' in "Python"
False
```

# Funkce nad řetězci

- Syntax: string.function()
- Nemění původní retězec
  - lower() - vrátí řetězec s malými písmeny místo velkých
  - upper() - vrátí řetězec s velkými písmeny místo malých

```
>>> reaction = "What The Hell?"
>>> reaction.lower()
'what the hell?'
>>> reaction.upper()
'WHAT THE HELL?'
>>> reaction
'What The Hell?'
```

# Funkce nad řetězci

- count(s) - spočítá výskyt řetězce *s* v daném řetězci

```
>>> yoda = "Do or do not... There is no try."
>>> yoda.count("o")
5
>>> yoda.count("no")
2
>>> yoda.count("do")
1
```

# Funkce nad řetězci

- find(s) - najde první index výskytu řetězce *s* v daném řetězci

```
>>> yoda = "Do or do not... There is no try."
>>> yoda.find("o")
1
>>> yoda.find("no")
9
>>> yoda.find("do")
6
>>> yoda.find("x")
-1
```

# Funkce nad řetězci

- replace(a, b) - vrátí nový řetězec, který v daném řetězci nahradí všechny výskyty řetězce a řetězcem b

```
>>> yoda = "Do or do not... There is no try."
>>> yoda.replace("o", "0")
'D0 0r d0 n0t... There is n0 try.'
>>> yoda . replace("no", "nooo")
'Do or do nooot... There is nooo try.'
>>> yoda.replace("There is no try.", "No try, there is.")
'Do or do not... No try, there is.'
>>> yoda.replace("anakin", "vader")
'Do or do not... There is no try.'
```

# Funkce nad řetězci

- split() - rozdělí řetězec podle mezer nebo specifikovaného oddělovače
  - Výsledkem je seznam řetězců
  - Oddělovač se v průběhu procesu zahazuje

```
>>> yoda = "Do or do not... There is no try."
>>> yoda.split()
['Do ', 'or ', 'do', 'not... ', 'There', 'is', 'no', 'try.']
>>> yoda.split('o')
['D', ' ', 'r d', ' n', 't... There is n', ' try.']
```

# Funkce nad znaky

- Znak je řetězec délky 1
- **ASCII**: http://www.ascii-code.com/
- ord(c), kde *c* je znak
  - Vrací celočíselný ASCII kód znaku
- chr(i), kde *i* je celé číslo
  - Vrací znak, jehož ASCII kód je *i*

```
>>> ord("A")
65
>>> chr(65)
'A'
```

# Úvodní úkol

- Napište funkci, která vám vrátí řetězec s písmeny uspořádanými pozpátku.
  - Použijte obě verze cyklu for
- Rozšíření v případě dostatku času
  - Mezi každá dvě písmena daného textu vložte dodaný text (jako nový parametr funkce)

# Úkoly

- Napište funkci, která vrátí nový řetězec, ve kterém bylo každé písmenko zdvojeno.
- Napište funkci, která dostane dva řetězce a vrátí ty znaky, které jsou na shodných pozicích stejné.
- Napište funkci, která vrátí, zda je řetězec palindromem. Palindromem je takové slovo či věta, která má při čtení v libovolném směru stejný význam, například nepotopen (můžete předpokládat, že mezery v řetězci nejsou).
- Každý znak A-Z má hodnotu 1-26 (diakritiku a velikost písmen pro tento příklad ignorujte, předpokládejte, že jsou všechna velká). Napište funkci, která spočítá a vrátí hodnotu vloženého řetězce (slova).